

День 11 • Локальное развертывание LLM, формирование закрытого контура

Обсуждаемые темы:

-  Обзор актуальных LLM (10 минут)
-  Тренды и ключевые модели для локального запуска (10 минут)
-  Вариант А: Локальный сервер (Ollama, OpenAI-совместимый API) (20 минут)
-  Вариант Б: Удаленный сервер (VPS + Docker + Nginx) (20 минут)
-  Заключение и рекомендации по выбору модели (5 минут)

Развертывание собственной LLM с HTTP API — отличный способ получить независимость от облачных провайдеров и полный контроль над своими данными.

В этом материале представлены:

-  Два детальных плана — для локального и удалённого развертывания
-  Обзор актуальных моделей — чтобы сделать осознанный выбор под ваше железо и задачи
-  Готовые способы поднять HTTP API в стиле OpenAI (без переписывания ваших инструментов)

Тренды 2026: специализация и эффективность

Эпоха «одной модели, чтобы править всеми» уходит в прошлое. Сейчас модели делятся на **два основных типа**:

1 Модели для глубоких рассуждений (Reasoning):

- Тратят больше времени на «размышления»
- Идеальны для математики, логики, сложного анализа кода
- Примеры: DeepSeek-R1, Qwen3 (режим thinking)

2 Модели для скорости и «действий» (Agentic):

- Оптимизированы для быстрого выполнения задач, работы с инструментами
- Идеальны для написания и отладки кода, агентных пайплайнов
- Примеры: Llama 4, Gemma 3, CodeQwen

⚙ Архитектура МоЕ: больше параметров — меньше затрат

Большинство современных моделей используют архитектуру **Mixture of Experts (MoE)**.

Как это работает:

- Модель имеет огромное количество суммарных параметров (например, 235B)
- При решении каждой конкретной задачи активируется лишь малая часть (например, 22B)
- Это экономит память и вычислительные ресурсы без потери качества

Практический результат: модели, которые ещё вчера требовали дата-центр, сегодня запускаются на одном GPU или даже на CPU.



Ключевые модели для локального развертывания

Модель	Разработчик	Сильные стороны	RAM/VRAM
DeepSeek-R1	DeepSeek	Математика, логика, программирование. Reasoning на основе RL	От ~8 GB (14B) до ~400 GB (671B MoE)
Qwen3 (235B-A22B)	Alibaba	Многоязычность, русский язык, два режима: thinking и fast	24–48 GB VRAM (активных 22B)
Gemma 3 / Gemma 3n	Google DeepMind	Отличное соотношение качества и размера. Легко на домашнем ПК	2B, 9B, 27B



Ключевые модели для локального развертывания

Модель	Разработчик	Сильные стороны	RAM/VRAM
Llama 4	Meta	Баланс reasoning/скорость. Огромное сообщество, поддержка Ollama	От 8B до 405B+
TinyLlama	Community	Проверка связей при минимуме ресурсов	1.1B (~1–2 GB RAM)

✓ Минимальные требования (ориентиры)

- **CPU:** 4+ ядер (быстрее — лучше; на CPU всё будет медленнее, но для 1B–7B возможно)
- **RAM:** 16 GB+ (комфортнее 32 GB, если планируете 14B+)
- **Диск:** 80–200 GB SSD (модели и кэши быстро “съедают” место)
- **GPU (опционально, но сильно ускоряет):**
 - 8–12 GB VRAM: обычно комфортно для 7B в квантах
 - 16 GB VRAM: хороший “народный минимум” для 14B в квантах
 - 24+ GB VRAM: заметно шире выбор (в т.ч. большие контексты/батчи)

💡 Практический старт без боли: начните с `gemma3:9b` или `qwen3:14b` и уже потом масштабируйтесь.

Что такое “кванты” и почему это важно

Большинство локальных запусков — это **квантованные модели** (меньше вес, меньше требований к памяти).

- **Q4 / Q5**: чаще всего “золотая середина” (скорость/качество)
- **Q8**: ближе к “почти FP”, но тяжелее
- **FP16/BF16**: обычно требуют заметно больше VRAM/RAM

Практическое правило: если модель **не помещается в VRAM**, часть уедет в RAM → будет сильно медленнее.

Два варианта развертывания

На выходе обоих вариантов — **эндпоинт, совместимый с форматом OpenAI**, что позволяет подключать любые инструменты: OpenWebUI, Continue.dev, Cursor и др.

	Вариант А: Локальный	Вариант Б: Удалённый (VPS)
Стек	Ollama (OpenAI-совместимый API)	Linux + Docker + (Ollama / vLLM) + Nginx
Сложность	● Низкая	● Средняя
Время	30–60 минут	1–2 часа
Подходит для	Разработка, тестирование	Продакшн, командная работа, 24/7 доступ

Вариант А: Локальный сервер

Шаг 1 — Установка движка (Ollama):

- Скачайте и установите Ollama с официального сайта для вашей ОС (Windows, macOS, Linux):
 <https://ollama.com>
- Ollama берёт на себя загрузку/хранение моделей, оптимизацию и запуск (в т.ч. поддержку GPU, где это возможно)

Требования к железу:

- Минимум: 8 GB RAM (для моделей 1B–7B на CPU)
- Рекомендуется: GPU с 8–16 GB VRAM (для моделей 7B–14B)
- Комфортно: 24+ GB VRAM (для моделей 30B+)

Вариант А: Загрузка модели

Шаг 2 — Загрузка и запуск модели:

Откройте терминал и выполните команду:

```
ollama run qwen3:14b
```

или

```
ollama run deepseek-r1:14b
```

Команда скачает модель и запустит с ней диалог в консоли для проверки. Для выхода используйте `/bye`.

Просмотр всех доступных моделей: <https://ollama.com/library>

Шаг 3 — Самый простой способ: OpenAI-совместимый API прямо из Ollama

После установки Ollama уже запускает фоновый сервер на `http://localhost:11434`.

Помимо “родного” API (`/api/generate` , `/api/chat`), у Ollama есть:

- ключевые маршруты: `/v1/chat/completions` , `/v1/completions` , `/v1/embeddings` , `/v1/models`

Проверка через curl (чат):

```
curl http://localhost:11434/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "qwen3:14b",  
  "messages": [{"role": "user", "content": "Почему небо голубое?"}]  
'
```

✓ Не требует написания кода — подходит для большинства новичков и инструментов.

Подключение инструментов “как к OpenAI”

Если инструмент поддерживает OpenAI API (SDK, LangChain, IDE-плагины), обычно достаточно указать:

- `base_url` : `http://localhost:11434/v1/`
- `api_key` : любая строка (часто обязательное поле, но Ollama может её не проверять)

Пример на Python (OpenAI SDK):

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:11434/v1/", api_key="ollama")
resp = client.chat.completions.create(
    model="qwen3:14b",
    messages=[{"role": "user", "content": "Сделай краткий план на день"}],
)
print(resp.choices[0].message.content)
```

Вариант А: HTTP API — Способ 2 (FastAPI)

Шаг 3 — Способ 2 (гибкий): прокси на FastAPI

Когда всё уже работает через `/v1/`, прокси нужен, если вы хотите:

- добавить **API key**, rate limit, аудит/логи
- единый домен/маршрутизацию (например, несколько бэкендов)
- привести ответы к нужному формату вашего приложения

🧩 Частая “грабля”: Docker ↔ Ollama и localhost

Если ваш прокси крутится в **Docker**, то `127.0.0.1` внутри контейнера — это **сам контейнер**, а не хост с Ollama.

- **Linux**: самый простой путь — `--network=host` (и тогда `127.0.0.1:11434` будет хостом)
- **Windows/macOS (Docker Desktop)**: используйте `http://host.docker.internal:11434/...`
- **Альтернатива**: выставить Ollama на сеть (например, `OLLAMA_HOST=0.0.0.0:11434`) и ограничить доступ фаерволом

Шаг 4 — Тестирование через Python:

```
import requests

response = requests.post(
    "http://localhost:8000/v1/completions",
    json={"prompt": "Напиши хокку про весну"}
)
print(response.json())
```

Или через curl:

```
curl -X POST http://localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Напиши хокку про весну"}'
```

Вариант Б: Удалённый сервер (VPS)

Шаг 1 — Подготовка сервера:

- Арендуйте VPS. Минимальные рекомендации: 4 CPU, 16 GB RAM, 20 GB SSD
- Для комфортной работы с моделями 7B–14B нужен GPU (NVIDIA T4 или лучше)
- Подключитесь по SSH и обновите систему:

```
sudo apt update && sudo apt upgrade -y
```

Шаг 2 — Установка Docker:

```
curl -fsSL https://get.docker.com -o get-docker.sh && sudo sh get-docker.sh
```

Для GPU в Docker дополнительно установите драйверы NVIDIA и `nvidia-container-toolkit` (и проверьте `nvidia-smi`).

Вариант Б: Настройка Ollama на сервере

Шаг 3 — Установка Ollama с поддержкой удалённого доступа:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Отредактируйте сервис, чтобы Ollama слушала все интерфейсы:

```
sudo systemctl edit ollama.service
```

Добавьте в открывшемся редакторе:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0:11434"
```

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
ollama pull deepseek-r1:14b
```

Вариант Б: Docker-образ для API

Шаг 4 — Структура проекта:

```
my-llm-server/  
├── Dockerfile  
├── main.py  
└── requirements.txt
```

Dockerfile :

```
FROM python:3.12-slim  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
COPY . .  
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Безопасность: что именно “нельзя делать”

- Не публикуйте Ollama (порт **11434**) в интернет без защиты.
- Наружу обычно выставляют **только** reverse proxy (**80/443**) и уже за ним ваш API (или OpenWebUI).
- Если вам нужен доступ из интернета — делайте это через **Nginx + SSL + авторизацию** (или через VPN / Tailscale / WireGuard).

Вариант Б: Деплой и тестирование

Шаг 5 — Сборка и запуск контейнера:

```
docker build -t llm-api .
docker run -d --name llm-api-container -p 8000:8000 --network=host llm-api
```

Для продакшна: настройте **Nginx** как reverse proxy (порты 80/443 → 8000) и добавьте **SSL-сертификаты** через Let's Encrypt.

Шаг 6 — Тестирование с удалённой машины:

```
curl http://<IP-вашего-сервера>:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Привет!"}'
```

⚠️ Настройте фаервол (UFW): открыть только 80/443, закрыть остальные порты.

Заключение и рекомендации

Выбор сценария:

-  Быстрый старт без кода → Ollama + OpenAI-совместимый `/v1/*` API на локальном ПК
-  Кастомизация и авторизация → добавьте FastAPI-прокси / Nginx поверх Ollama
-  Продакшн на VPS → Docker + Nginx + SSL, рассмотрите OpenWebUI (веб-интерфейс + RAG), либо изоляцию через VPN

Выбор модели:

-  Математика и логика → DeepSeek-R1
-  Универсальная с русским языком, два режима (thinking/fast) → Qwen3
-  Много кода → CodeQwen или DeepSeek-Coder
-  Слабое железо, тестирование → Gemma 3 (2B–9B) или TinyLlama

Вы получили инструменты — но опыт начинается с практики... 🔥

🎓 **Исследуйте** — пробуйте разные модели, экспериментируйте с параметрами

🔍 **Углубляйтесь** — RAG, fine-tuning, квантование открывают новые возможности

🚀 **Создавайте** — даже простой локальный API лучше теории

Это невозможно, пока вы это не сделаете!

Успехов!